

DEDICATED TO INNOVATION

www.bacancy.com





**Customer Satisfaction
Is Our Highest Priority**



Employee Matters



In this session

What we are learning in this session

- ▶ Arrays and Collections
- ▶ Methods and Functions
- ▶ Exception Handling

Let's Start

Arrays and Collections

□ Array Basics

- Fixed-size collection of elements
- Elements are of the same type
- Accessed by index

□ Lists and Dictionaries

- Lists: Dynamic-sized collection
- Dictionaries: Key-value pairs for efficient lookups

Methods and Functions

- ▶ Methods are generally the block of codes or statements in a program that gives the user the ability to reuse the same code which ultimately saves the excessive use of memory, acts as a time saver and more importantly, it provides a better readability of code. So basically, a method is a collection of statements that perform some specific task and return the result to the caller. A method can also perform some specific task without returning anything.

- ▶ **Method**

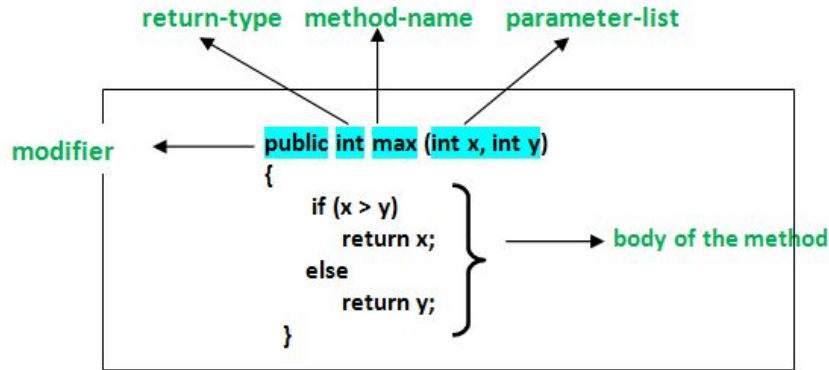
Method declaration means the way to construct method including its naming.

- ▶ **Syntax:**

`<Access_Modifier> <return_type> <method_name>([<param_list>])`

Declaration:

Methods and Functions



- ▶ In C# a method declaration consists of the following components as follows :
- ▶ **Modifier:** It defines access type of the method i.e. from where it can be accessed in your application. In C# there are Public, Protected, Private access modifiers.
- ▶ **Name of the Method:** It describes the name of the user defined method by which the user calls it or refer it. Eg. GetName()
- ▶ **Return type:** It defines the data type returned by the method. It depends upon user as it may also return void value i.e return nothing

Methods and Functions

- ▶ **Body of the Method:** It refers to the line of code of tasks to be performed by the method during its execution. It is enclosed between braces.
- ▶ **Parameter list:** Comma separated list of the input parameters are defined, preceded with their data type, within the enclosed parentheses. If there are no parameters, then empty parentheses () have to use out.

- ▶ **Method**

Signature:

Method Signature is defined by mainly two parameters(number of parameters, type of the parameters and order of the parameters), One of them is Method Name and second one is its Parameter list.

- ▶ **Method**

Signature:

Method Signature is defined by mainly two parameters(number of parameters, type of the parameters and order of the parameters), One of them is Method Name and second one is its Parameter list.

- ▶ **Method**

Naming:

Name of a method or a function in any programming language whether in C++ or Java or C# holds great importance and is mainly used in order to call that method for its execution. For example, findSum, computeMax, setX and getX etc.

Methods and Functions

- ▶ There are certain pre-defined rules for naming methods which a user should follow:
 - The method name must be some kind of Noun or a verb.
 - It's naming should be done in such a way that it must describe the purpose of that method.
 - The first letter of the method name can be either a small letter or a Capital letter, however, it is recommended to use the capital one.

- ▶ **Advantages of using the Methods:**

There are many advantages of using methods. Some of them are listed below:

- It makes the program well structured.
- Methods enhance the readability of the code.
- It provides an effective way for the user to reuse the existing code.
- It optimizes the execution time and memory space.

Exception Handling

- **Exception** occurs during execution of program and it breaks normal flow of program execution. If it is not handled It can cause the application to stop working.
- C# provides built-in support to handle these exceptions using try-catch blocks.
- C# has its own set of specific exceptions which can be used in an application.
- Some of them are:
 NullReferenceException,
 ArgumentOutOfRangeException,
 InvalidCastException,
 FileNotFoundException,
 InvalidOperationException etc.

Exception Handling

```
try{  
    // Any suspected code that can raise exceptions should be put inside this block.  
}  
catch {  
    // This block is an exception handler block where some action such as logging and auditing an  
    exception can be performed  
}  
finally {  
    // his block will always be executed whether an exception raised or not. Usually, this block should  
    be used to release resources, e.g., to close any stream or file objects that were opened in the try  
    block.  
}
```

Exception Handling

```
class Program {  
    static void Main(string[] args) {  
        try  
        {  
            Console.WriteLine("Enter your number: ");  
            int num = Convert.ToInt32(Console.ReadLine());  
            Console.WriteLine($"Result is ${Math.Sqrt(num)}");  
            Console.ReadKey();  
        }  
        catch (Exception ex) {  
            Console.WriteLine($"There is something wrong, please look at this message: {ex.Message}");  
            Console.ReadKey();  
        }  
    }  
}
```

//input from User is twenty instead of 20

Thank you

Self Learning

- ▶ Collections and LINQ
- ▶ Control Flow
- ▶ Enums and Structs

Collections and LINQ

▶ What is a Collection?

In programming, a **collection** is a data structure used to store, manage, and manipulate multiple items (objects) as a single unit. Collections provide a way to group items, often of the same type, for operations like adding, removing, iterating, and searching.

Collections and LINQ

► Dictionary<TKey, TValue>

- A collection of key-value pairs where keys must be unique.
- **Use Case:** Efficient lookups by key.

```
var dictionary = new Dictionary<int, string>
{
    { 1, "One" },
    { 2, "Two" },
    { 3, "Three" }
};

if (dictionary.TryGetValue(2, out var value))
{
    Console.WriteLine($"Key 2: {value}"); // Output: Key 2: Two
}
```

Collections and LINQ

▶ HashSet<T>

- A collection of unique elements that does not allow duplicates.
- **Use Case:** Fast operations like checking existence, adding, or removing elements.

```
var hashSet = new HashSet<int> { 1, 2, 3, 3 }; // Duplicates are ignored

hashSet.Add(4);
hashSet.Remove(2);

Console.WriteLine(string.Join(", ", hashSet)); // Output: 1, 3, 4
```

Collections and LINQ

► LINQ Queries (Language Integrated Query)

Filtering (Where)

- **Use Case:** Filters elements based on a condition.

```
var numbers = new List<int> { 1, 2, 3, 4, 5, 6 };  
var evenNumbers = numbers.Where(n => n % 2 == 0);
```

```
Console.WriteLine(string.Join(", ", evenNumbers)); // Output: 2, 4, 6
```

Ordering (OrderBy/OrderByDescending)

Use Case: Orders elements in ascending or descending order.

```
var names = new List<string> { "John", "Alice", "Bob" };  
var sortedNames = names.OrderBy(name => name);
```

```
Console.WriteLine(string.Join(", ", sortedNames)); // Output: Alice, Bob, John
```

Control Flow

Conditional Statements

- If-else
- else-if
- Switch-case

Jump Statements

- Break
- Continue
- Return
- goto

Loops

- For
- ForEach
- While
- Do-While

Enums and Structs

- An enum type (or enumeration type) is a value type defined by a set of constants of with their underlying numeric type.
- To define an enumeration type, use the enum keyword and specify the names of enum members.

Enums and Structs

```
public enum Season
{
    Spring,
    Summer,
    Autumn,
    Winter
}
```

```
public class EnumConversionExample
{
    public static void Main()
    {
        Season a = Season.Autumn;
        Console.WriteLine($"Integral value of {a} is {(int)a}");

        var b = (Season)1;
        Console.WriteLine(b); // output: Summer

        var c = (Season)4;
        Console.WriteLine(c); // output: 4
    }
}
```

// output: Integral value of Autumn is 2